

Data-Efficient Vulnerability Detection via Semantic Image Generation and Adversarial Reprogramming

Anonymous Authors

Abstract—Software vulnerabilities pose critical threats to modern software systems. While recent deep learning based approaches have shown encouraging performance, they typically require substantial data for effective model training. To this end, we propose SIGAR, a data-efficient vulnerability detection framework that combines Semantic Image Generation with Adversarial Reprogramming. Specifically, SIGAR first constructs the program dependency graphs of the functions and converts them into multi-channel code semantic images that encode both fine-grained semantics and structural dependencies. These images are then processed by a vision model pre-trained on ImageNet, which is adapted to the vulnerability detection task through adversarial reprogramming using a lightweight input transformation and a straightforward output label mapping, without updating the model backbone. By restricting optimization to a small set of perturbation parameters, SIGAR achieves efficient learning with reduced data, while effectively leveraging the representational power of pre-trained models. Extensive experiments on three real-world datasets demonstrate that SIGAR consistently surpasses 15 well-recognized baseline models, achieving absolute average performance gains of 5.06% in accuracy and 8.11% in F1, while requiring significantly fewer trainable parameters.

Index Terms—vulnerability detection, code semantic images, adversarial reprogramming, data-efficient training

I. Introduction

In recent years, the software security landscape has experienced a rapid surge in both the number of reported vulnerabilities and the severity of associated threats. By 2026, the total number of publicly disclosed Common Vulnerabilities and Exposures (CVE) has exceeded 320,000, with approximately 50,000 new entries reported in 2025 alone. Given the pervasive nature of software vulnerabilities and their significant impact, extensive efforts from both academia and industry have been devoted to developing effective detection techniques [1], [2]. In particular, the rapid advancement of deep learning (DL) has enabled the development of data-driven approaches [3], which leverage powerful representation learning capabilities for automated identification of potential vulnerabilities.

Within DL-based vulnerability detection, models are typically trained on labeled datasets to learn semantic representations and capture vulnerability-specific patterns [4]. Early methods primarily adopt a training-from-scratch paradigm, where code is modeled as sequences or structured graphs and processed using architectures such as RNNs [5] or graph neural networks [6]–[8]. Subsequently, pre-trained models (PTMs), such as CodeBERT [9], have been introduced and fine-tuned on vulnerability datasets to exploit their code understanding capabilities and improve detection performance [10], [11]. More recently, the rise of large language models (LLMs)

has further driven prompt-based vulnerability detection [12]–[14], where inference is guided by carefully designed prompts.

Despite the notable progress in DL-based vulnerability detection researches, following critical challenges remain:

- **Effectiveness under limited data.** Compared to the abundance of benign code, real-world vulnerable samples are inherently scarce [15]–[17]. This scarcity stems from the limited number of publicly disclosed vulnerabilities, their dispersion across diverse vulnerability types and programming languages, posing a fundamental challenge to data-driven models. Both training-from-scratch methods and PTM-based approaches typically involve a large number of trainable parameters and thus generally require substantial and diverse training data to achieve strong performance. For instance, fine-tuning CodeBERT entails updating approximately 220M parameters, which becomes difficult to optimize effectively when vulnerable samples are limited [18], [19].
- **High computational cost.** While LLMs provide a promising paradigm for prompt-based vulnerability analysis, their performance often remains inferior to models specifically trained or fine-tuned for the task [20]–[23]. Although further fine-tuning on vulnerability datasets can enhance their effectiveness [20], [24], [25], it incurs substantial computational and memory overhead, limiting practicality. Moreover, the large model size required for local deployment and the high token costs associated with API-based usage further hinder their widespread adoption, particularly in resource-constrained settings.

To deal with these limitations, we investigate a paradigm based on adversarial reprogramming [?], [26], which enables the adaptation of pre-trained models to new tasks via lightweight input transformations and output remapping. Specifically, a set of learnable universal perturbations is applied to the input, allowing a model originally trained in a source domain to be redirected toward possibly a completely different target task [27]–[29] without altering its internal parameters. As the training process involves only a small number of learnable perturbation parameters, which are significantly fewer than the full model weights, this approach greatly reduces computational overhead and dependence on large-scale labeled data, while still leveraging the powerful feature extraction capabilities of pre-trained models.

Building upon this paradigm, we propose SIGAR, a data-efficient vulnerability detection approach that integrates Semantic Image Generation with Adversarial Reprogramming.

Specifically, SIGAR first transforms the program dependence graph (PDG) of a function into a multi-channel code semantic image by combining semantic embeddings derived from Sent2Vec with complementary structural embeddings generated by Node2Vec, DeepWalk, and LINE, thereby capturing both fine-grained semantic information and rich structural dependencies. Leveraging this representation, SIGAR repurposes a vision model pre-trained on ImageNet for vulnerability detection via adversarial reprogramming, where the semantic images are fed into a frozen backbone augmented with a learnable input transformation, while a label mapping converts the original ImageNet outputs into binary vulnerability predictions. By optimizing only a small set of perturbation parameters while keeping the backbone model unchanged, SIGAR achieves effective and computationally efficient model training under limited-data settings, without requiring costly graph neural network training or extensive model fine-tuning.

The main contributions are summarized as follows:

- We introduce a data-efficient vulnerability detection framework that reformulates program analysis as a visual learning problem and leverages adversarial reprogramming to repurpose pre-trained vision models for cross-domain adaptation without fine-tuning.
- We propose a new method to transform the code into multi-channel code semantic images, capturing both fine-grained semantics and rich structural dependencies.
- We design a lightweight reprogramming strategy that adapts pre-trained vision models for vulnerability detection by optimizing only a small set of input perturbation parameters, significantly reducing data requirements while maintaining strong detection capability.
- Extensive evaluations on three real-world datasets demonstrate that SIGAR delivers superior performance over existing methods, with average gains of 5.06% in accuracy and 8.11% in F_1 . It also shows clear advantages in low-data settings, achieving an average improvement of 8.0% in F_1 over the baseline methods. To support reproducibility and future research, we release our implementation at <https://anonymous.4open.science/r/AdvRVD-9185>.

The remainder of this paper is organized as follows. Section II reviews related studies. Section III introduces the necessary background. Section IV presents the design of SIGAR. Section V reports the experimental results. Section VI discusses limitations, and Section VII concludes the paper.

II. Related Work

A. DL-based Vulnerability Detection

Deep learning has significantly advanced vulnerability detection [3], [4], [30] by enabling models to automatically capture latent vulnerability patterns embedded in source code. A large body of prior work adopts a training-from-scratch paradigm, which can be broadly categorized based on code representation into sequence-based and graph-based methods. Sequence-based methods [31] convert code into token sequences, and employ neural architectures such as CNNs [32]

and LSTMs [33] to learn semantic features for vulnerability prediction. However, these methods are often limited in modeling the complex structural dependencies inherent in programs. Thus, subsequent research [7], [8], [34] has increasingly focused on graph-based representations, which encode richer structural and semantic information. These approaches model code using structures such as CFGs [35], [36], ASTs [37], [38], PDGs [39], as well as other composite or derived graph forms [40], [41], such as CPG [42], FG [43], and HeVG [44].

Another line of work leverages pre-trained models [10], [45] or large language models (LLMs) [12], [46]. For example, Kalouptoglou et al. [47] fine-tune CodeBERT and CodeGPT for vulnerability detection, while EPVD [11] enhances CodeBERT with CNN-based encoding of syntactic paths from syntax-CFGs. More recent studies [12], [25], [48]–[51] explore prompt-based vulnerability detection using LLMs such as ChatGPT and LLaMA. While some studies [13], [14] report promising results of prompting LLMs for vulnerability detection, particularly when provided with enriched contextual code [13]; considerable amount of studies [20]–[23], [52] also reveal that LLMs still face significant challenges in effectively identifying vulnerabilities with zero-shot or few-shot prompting strategies. Conducting further LLM fine-tuning with vulnerability data [20], [24], [25], [53] seems helpful to improve their later prompt-based vulnerability inference performance. Nevertheless, the practical deployment of LLM-based vulnerability detectors remains challenging. The substantial model size required for local deployment or the high API costs hinder their widespread adoption, particularly in resource-constrained environments.

B. Vulnerability Detection Considering Data Scarcity

Despite ongoing efforts to construct real-world vulnerability datasets, high-quality labeled vulnerable samples remain limited. To alleviate this issue and improve the effectiveness of data-driven models, some studies [54], [55] employ data augmentation techniques at the representation level to increase sample diversity. For instance, MPDA [54] introduces multiple strategies, including SMOTE, GAN-based generation, and fuzzing-based sampling, to synthesize additional training data. Another line of research focuses on automated vulnerability code generation [16], [18], [56]. VulGen [57] combines pattern mining with DL-based localization to inject vulnerabilities into code, while VGX [15] enhances this process with mutation rules and a semantics-aware Transformer to identify appropriate injection contexts. More recent approaches, such as GVI [58], VulScribeR [17], and VICD [59], leverage curated prompts to guide LLMs in generating new vulnerable samples through mutation, injection, and extension strategies. In contrast to these methods centered on data augmentation or synthetic sample generation, our approach adopts an adversarial reprogramming paradigm to facilitate effective learning under limited data conditions. Notably, these strategies are complementary, synthetically generated samples can be in-

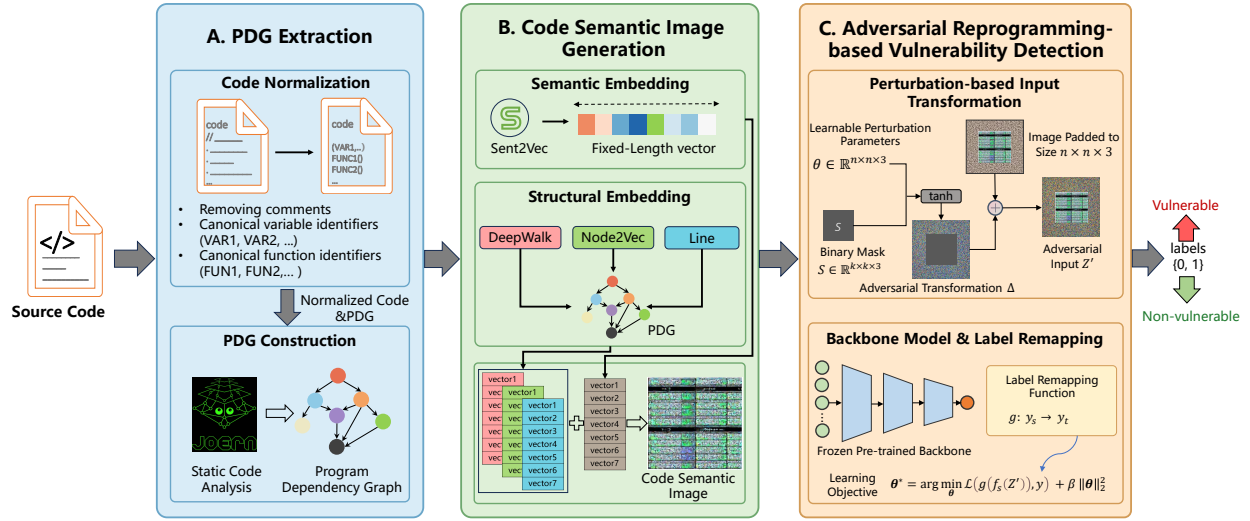


Fig. 1: The overall workflow of SIGAR

incorporated into few-shot training settings, further enhancing overall model performance.

III. Preliminaries

Adversarial reprogramming [26], [27], [60] is a technique that enables a pre-trained neural network to be adapted to a new task without altering its original parameters. Unlike adversarial attacks, which aim to degrade model performance or induce incorrect outputs, adversarial reprogramming repurposes a model to accomplish a different objective, sharing a similar goal with transfer learning. However, the two paradigms differ fundamentally: transfer learning typically requires parameter fine-tuning and assumes a certain level of domain similarity, whereas adversarial reprogramming keeps the backbone model fixed and instead learns lightweight transformations, allowing effective cross-domain adaptation even when the source and target tasks are substantially different.

Specifically, adversarial reprogramming introduces an input transformation function and an output mapping function to bridge the source and target tasks [27]. During training, only the transformation parameters are optimized, while the original model remains unchanged, resulting in improved efficiency. Formally, given a pre-trained source model $s(\cdot)$ that maps inputs $\mathbf{x}_s \in \mathcal{X}_s$ to labels $y_s \in \mathcal{Y}_s$, a target task model $t(\cdot)$ is constructed by incorporating an input transformation $f_s(\mathbf{x}_t, \theta)$ and a label mapping $f_t(\cdot)$ for target inputs $\mathbf{x}_t \in \mathcal{X}_t$. The transformation f_s projects target inputs into the source domain through a learnable perturbation θ , while f_t converts the source model's outputs into the corresponding target labels.

Owing to its limited number of trainable parameters, adversarial reprogramming reduces both data requirements and computational overhead, while still leveraging the representational power of pre-trained models to capture fine-grained semantic patterns.

IV. Design of SIGAR

This section details the design of SIGAR. As illustrated in Figure 1, the framework comprises three main stages: PDG

extraction, code semantic image generation, and vulnerability detection based on adversarial reprogramming.

A. PDG Extraction

Given the critical role of data and control dependencies in capturing program semantics and execution behavior, we adopt the program dependence graph as the initial code representation to operate on. The PDG has been widely utilized in prior DL-based vulnerability detection studies [5], [39], [61] due to its ability to explicitly model both data and control relationships among code elements.

Prior to PDG construction, we apply a code normalization procedure to improve the robustness of SIGAR against common syntactic variations, such as identifier renaming. By standardizing variable and function names, we reduce token diversity and discourage the model from relying on superficial naming patterns, while also mitigating out-of-vocabulary issues and improving generalization. In addition, comments are removed as they do not affect program execution and may introduce irrelevant or even Interfering information. Specifically, we employ a three-step normalization process: (1) removing all comments from the source code; (2) replacing user-defined variables with canonical identifiers (e.g., VAR1, VAR2) based on their order of appearance; and (3) replacing user-defined function names with canonical identifiers (e.g., FUN1, FUN2) following the same scheme.

Following code normalization, we utilize Joern, a widely used static analysis framework, to construct the PDG for each function. Joern supports multiple programming languages and can effectively handle incomplete or non-compilable code, making it suitable for practical scenarios where code snippets are often partial or imperfect. By leveraging its robust parsing and graph construction capabilities, we obtain consistent and reliable PDG representations across diverse inputs. Figure 2 illustrates an example of the normalization process together with the resulting PDG. In this graph, each node corresponds to a statement (i.e., a line of code), while edges explicitly

capture data dependencies (such as variable definitions and uses) and control dependencies (such as branching conditions). This representation effectively preserves both structural and semantic relationships within the code, serving as a strong foundation for subsequent vulnerability analysis.

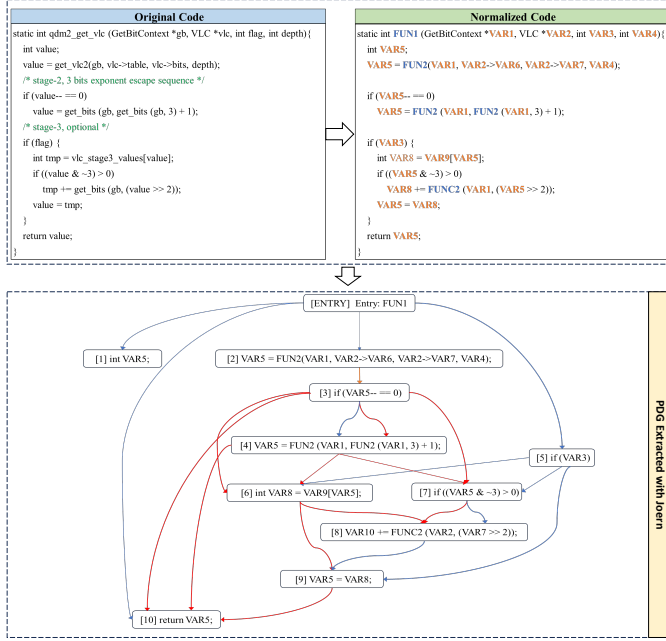


Fig. 2: An example of code normalization and extracted PDG.

B. Code Semantic Image Generation

Given a graph representation of the function, a straightforward approach is to apply graph neural networks (GNNs) to learn structural features, or to perform program slicing based on data and control dependencies in the PDG, followed by sequence modeling (e.g., RNNs) for vulnerability detection. Inspired by prior work such as VulCNN, which demonstrates the effectiveness of transforming PDGs into image-like representations for efficient processing with CNNs, SIGAR adopts a similar strategy by converting the PDG into a code semantic image.

Given a graph representation of a function, conventional approaches typically apply graph neural networks (GNNs) [6], [40] to model structural dependencies or perform program slicing [5] followed by sequence models for vulnerability detection. In this work, SIGAR instead organizes complementary semantic and structural signals into a unified representation that can be adapted to a large-scale pre-trained model. Inspired by image-based vulnerability detection methods such as VulCNN [61], we construct a multi-channel code semantic image from the PDG. This transformation organizes node-level semantic and structural features into a structured two-dimensional representation compatible with an ImageNet-pre-trained vision backbone. Through adversarial reprogramming, SIGAR reuses the backbone’s pre-trained feature extraction capability for the vulnerability detection task.

To capture both semantic and structural information, SIGAR represents each PDG node using complementary embeddings. Specifically, we first derive a semantic embedding for each node by applying Sent2Vec [62] to its corresponding code statement. Each node, representing a line of code, is treated as a sentence and encoded using a pre-trained Sent2Vec model, yielding fixed-length vectors that capture compositional semantic information.

To further model structural dependencies, we employ three unsupervised graph embedding methods, including DeepWalk [63], Node2Vec [64], and LINE [65]. These methods generate node embeddings that capture complementary aspects of the PDG topology, including local proximity, biased neighborhood exploration, and first- and second-order relationships.

Finally, we integrate these embeddings to construct a code semantic image. The structural embeddings, i.e., DeepWalk, Node2Vec, and LINE, are mapped to three channels, analogous to RGB channels in images, and enriched with Sent2Vec-based semantic features. This multi-view fusion enables the model to capture diverse semantic and structural characteristics of the program. The overall process of transforming functions into code semantic images is summarized in Algorithm 1.

Algorithm 1 Code Semantic Image Generation

Require: Function source code $\mathcal{F}$

Ensure: Code semantic image $\mathcal{I}$

- 1: $\hat{\mathcal{F}} \leftarrow \text{Normalize}(\mathcal{F})$
- 2: $\mathcal{G} = (V, E) \leftarrow \text{ConstructPDG}(\hat{\mathcal{F}})$
- 3: $\mathbf{E}^{(s)} \leftarrow \text{Sent2Vec}(V)$ ▷ Semantic embeddings
- 4: $\mathbf{E}^{(d)} \leftarrow \text{DeepWalk}(\mathcal{G})$
- 5: $\mathbf{E}^{(n)} \leftarrow \text{Node2Vec}(\mathcal{G})$
- 6: $\mathbf{E}^{(l)} \leftarrow \text{LINE}(\mathcal{G})$
- 7: **for** each node $v_i \in V$ **do**
- 8: $\mathbf{c}_i^{(1)} \leftarrow \mathbf{E}_i^{(s)} \oplus \mathbf{E}_i^{(d)}$
- 9: $\mathbf{c}_i^{(2)} \leftarrow \mathbf{E}_i^{(s)} \oplus \mathbf{E}_i^{(n)}$
- 10: $\mathbf{c}_i^{(3)} \leftarrow \mathbf{E}_i^{(s)} \oplus \mathbf{E}_i^{(l)}$
- 11: **end for**
- 12: $\mathbf{C}^{(1)} \leftarrow \text{Stack}(\{\mathbf{c}_i^{(1)}\}_{i=1}^{|V|})$
- 13: $\mathbf{C}^{(2)} \leftarrow \text{Stack}(\{\mathbf{c}_i^{(2)}\}_{i=1}^{|V|})$
- 14: $\mathbf{C}^{(3)} \leftarrow \text{Stack}(\{\mathbf{c}_i^{(3)}\}_{i=1}^{|V|})$
- 15: $\mathcal{I} \leftarrow \text{ComposeImage}(\mathbf{C}^{(1)}, \mathbf{C}^{(2)}, \mathbf{C}^{(3)})$
- 16: **return** $\mathcal{I}$

C. Adversarial Reprogramming-based Vulnerability Detection

Given the constructed code semantic images, SIGAR avoids training a convolutional neural network model from scratch and instead leverages adversarial reprogramming to more effectively deal with the scarcity of well-labeled vulnerability data. In particular, we repurpose a pre-trained vision model to perform vulnerability detection by introducing a learnable input transformation and an output label mapping, enabling efficient cross-domain adaptation.

Given the constructed code semantic images, SIGAR avoids training a model from scratch and instead employs adversarial reprogramming to better address the scarcity of well-labeled

vulnerability data. Specifically, SIGAR adopts an ImageNet-pre-trained vision backbone as the frozen source model. The semantic image representation organizes heterogeneous semantic and structural code features into a regular multi-channel input compatible with the vision backbone, enabling SIGAR to reuse its pre-trained feature extraction capability. During adaptation, the backbone remains frozen, and only a lightweight adversarial program is optimized for the vulnerability detection task.

1) *Perturbation-based Input Transformation*: As discussed in Section III, the core idea of adversarial reprogramming is to learn a universal perturbation that transforms inputs from the target domain into a form compatible with a source-domain model. In our setting, we utilize a vision model pre-trained on ImageNet for image classification and reprogram it for vulnerability detection.

Formally, we define the adversarial transformation as :

$$\Delta = \phi(\theta \circ \mathbf{S}) \quad (1)$$

where $\theta \in \mathbb{R}^{n \times n \times 3}$ denotes the learnable perturbation parameters, and n signifies the boundary size of this perturbation region. $\mathbf{S}$ is a binary mask that preserves the code semantic image region, $\circ$ represents element-wise multiplication, and $\phi(\cdot)$ is a bounded activation function (e.g., tanh) to constrain the perturbation magnitude within a predefined range of $(-1, 1)$ to stabilize training.

Let $\mathbf{Z} \in \mathbb{R}^{n \times n \times 3}$ denote the resized (or padded) code semantic image derived from the original input $\mathbf{X} \in \mathbb{R}^{k \times k \times 3}$, where $k < n$. The adversarial input is then constructed as:

$$\mathbf{Z}' = \mathbf{Z} + \Delta \quad (2)$$

This ensures that the resulting transformed inputs match the dimensionality and distribution expected by the pre-trained model. During training, the parameters of the backbone model remain fixed, and only θ is optimized, significantly reducing the number of trainable parameters.

2) *Label Remapping*: To align the output space of the source model with the target task, we construct a mapping from the ImageNet label space $\mathcal{Y}_s = \{0, \dots, 999\}$ to the binary vulnerability label space $\mathcal{Y}_t = \{0, 1\}$. Following prior practices [27], [29], [60], which demonstrate that hard-coded mappings can facilitate faster and more stable convergence, we adopt a simple deterministic mapping function $g : \mathcal{Y}_s \rightarrow \mathcal{Y}_t$, ~~where two ImageNet classes are randomly selected to correspond to vulnerable and non-vulnerable labels, respectively.~~ In our implementation, the first two ImageNet classes are used to represent the non-vulnerable and vulnerable labels, respectively, and the same mapping is kept unchanged across all experiments.

The learning objective is to optimize the perturbation parameters such that the reprogrammed model performs well on the vulnerability detection task. Formally, we solve:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(g(f_s(\mathbf{Z}')), y) + \beta \|\theta\|_2^2 \quad (3)$$

where $f_s(\cdot)$ is the fixed pre-trained model, $y \in \mathcal{Y}_t$ is the ground-truth label, $\mathcal{L}(\cdot)$ denotes the cross-entropy loss, and β is a regularization coefficient.

V. Experiments and Results Analysis

To assess the effectiveness of SIGAR, we conduct a series of experiments designed to answer the following research questions (RQs):

- **RQ1: Overall Detection Performance.** How does SIGAR perform compared with existing approaches in identifying vulnerabilities?
- **RQ2: Performance under Limited Data.** How well does SIGAR perform under limited training data compared to baseline methods?
- **RQ3: Effect of Adversarial Reprogramming.** To what extent do the performance gains of SIGAR originate from adversarial reprogramming rather than the semantic image representation or the use of a pre-trained vision backbone?
- **RQ4: Effect of Backbone and Parameter Scale.** How do different backbone models and perturbation parameter sizes influence the performance of SIGAR?
- **RQ5: Effect of Semantic Image Construction.** How does SIGAR perform with alternative semantic image construction strategies?

A. Experimental Setup

1) *Datasets*: To evaluate the effectiveness of SIGAR and answer the research questions, we conduct experiments on three widely used real-world vulnerability detection datasets. Table I provides an overview of their statistics, and we briefly describe each dataset below:

- **D2A** [66] is a vulnerability dataset released by IBM, covering six widely used open-source projects, including FFmpeg, httpd, Libav, LibTIFF, Nginx, and OpenSSL. It is constructed via differential analysis to label vulnerabilities identified by static analysis tools. In this study, we use its function-level subset.
- **Devign** [67] contains function-level C/C++ code collected from two widely used open-source projects, QEMU and FFmpeg. The dataset has undergone multiple rounds of manual annotation and verification by security experts to ensure label quality.
- **Reveal** [6] is another real-world dataset derived from the Linux Debian kernel and Chromium projects. It features a highly imbalanced distribution between vulnerable and non-vulnerable samples, reflecting more realistic scenarios compared to earlier datasets.

TABLE I: Summary of the datasets.

Dataset	#Vulnerable	#Non-Vulnerable	Vul_Ratio
D2A	2,795	2,444	53.35%
Devign	12,460	14,858	45.61%
Reveal	2,240	20,494	9.85%

2) *Baseline models*: We compare SIGAR with fifteen representative baseline approaches across three categories: (a) six training-from-scratch methods, including TokenLSTM, TokenCNN, SySeVR, Reveal, VulCNN, and TrVD; (b) five pre-trained model (PTM) fine-tuning methods, including CodeBERT, UniXCoder, GraphCodeBERT, LineVul, and EPVD; and (c) four prompt-based methods built upon widely used large language models (LLMs), namely DeepSeek-V3, Qwen3-14B, GPT-4o, and GPT-5. A brief overview of these baselines is provided below.

- **(a) Six Training-From-Scratch Methods:**

- **TokenLSTM** and **TokenCNN**: two sequence-based models that have been widely used as comparison baselines in previous studies, where the functions’ token sequences are encoded using either a BiLSTM or a TextCNN, followed by an MLP classifier for vulnerability prediction.
- **Reveal** [6]: a graph-based approach that constructs Code Property Graphs (CPGs) and leverages Gated Graph Neural Networks (GGNNs) to learn features for vulnerability detection.
- **SySeVR** [5]: a slicing-based method that derives semantic and syntactic program slices from PDGs and models them using RNNs to capture vulnerability patterns.
- **VulCNN** [61]: an image-based method that transforms PDGs into images using node embeddings combined with centrality measures, which are subsequently processed by a CNN for vulnerability detection.
- **TrVD** [37]: a tree-based model that extracts features from AST subtrees using an attention-enhanced RNN, followed by a Transformer-based feature aggregator for final prediction.

- **(b) Five PTM Fine-Tuning based Methods:**

- **CodeBERT** [9], **GraphCodeBERT** [68], and **UniXCoder** [69]: three representative pre-trained code models, which are fine-tuned on the vulnerability datasets to adapt them for vulnerability detection.
- **LineVul** [10]: a sequence-based approach that applies BPE tokenization to generate sub-token sequences, which are embedded with positional information and fed into CodeBERT for feature extraction and vulnerability classification.
- **EPVD** [11]: a method that constructs syntax-based control flow graphs and decomposes them into multiple paths. It combines CodeBERT and CNN with intra- and inter-path attention to learn path-level representations for vulnerability prediction.

- **(c) Four LLM Prompting-based Methods:** we conduct vulnerability inference using four representative large language models: **Qwen3-14B**, **DeepSeek-V3**, **GPT-4o**, and **GPT-5**. Qwen3-14B is deployed locally on our GPU server, while the other three models are accessed through official API services. All models are evaluated under a zero-shot setting using the vulnerability detection prompt

proposed in prior work [20].

3) *Implementation and Experimental Settings*: We implement SIGAR in Python using the PyTorch framework. For code semantic image construction, we utilize Joern to extract program dependence graphs (PDGs), and configure the embedding modules to produce 128-dimensional node representations, resulting in multi-channel images of size $128 \times 128 \times 3$. For model reprogramming, we directly use pre-trained vision models sourced from TIMM¹ and utilize the ImageNet-pretrained vit_base_patch16_384 as our default backbone.

To ensure fair comparison, SIGAR and all baseline methods in categories (a) and (b) are trained on identical data splits, with each dataset partitioned into training, validation, and test sets following an 8:1:1 ratio. In contrast, LLM-based methods in category (c) are evaluated via direct prompting on the test set. All models require training are optimized using Adam with an initial learning rate of 1×10^{-3} , decayed by a factor of 0.96 every two epochs. We employ early stopping based on validation performance, where the training data are shuffled at each epoch, and training terminates if the validation accuracy does not improve for 10 consecutive epochs. The model achieving the best validation F1 score is selected for final evaluation on the test set. All experiments are conducted on a Linux server equipped with two 2.1GHz Intel Silver CPUs, 128GB RAM, and two NVIDIA RTX 3090 GPUs.

For performance evaluation metrics, we report accuracy, precision, recall, and F1-score, following common practice in DL-based vulnerability detection studies.

B. Experimental Results

1) *RQ1: Overall Detection Performance Against Baselines*: Table II presents the vulnerability detection performance of all methods on the three datasets. As shown, although the performance varies across the datasets, SIGAR consistently achieves the highest F1 scores and attains the best accuracy in most cases. Notably, on the Reveal dataset that is characterized by severe class imbalance, SIGAR demonstrates strong robustness, achieving the highest F1 score of 34.34%, outperforming all baselines by a significant average margin of 8.12%. SIGAR effectively improves the detection of vulnerable samples without being biased toward the dominant class, thereby maintaining a better balance between precision and recall under diverse data distributions.

Specifically, in comparison with training-from-scratch approaches, SIGAR demonstrates notable improvements, with average absolute gains of 3.56% in accuracy and 7.30% in F1 score. These improvements can be attributed to the effective reuse of pre-trained knowledge through adversarial reprogramming, which enhances feature extraction without requiring extensive parameter updates.

When compared with PTM-based fine-tuning methods, SIGAR continues to deliver superior performance, achieving average improvements of 1.04% in accuracy and 5.64% in F1

¹<https://huggingface.co/timm>

Method \ Dataset	D2A				Devign				Reveal			
	Acc	F1	Pre	Rec	Acc	F1	Pre	Rec	Acc	F1	Pre	Rec
BiLSTM	54.19	55.02	55.85	54.22	53.26	41.72	45.52	38.50	86.10	26.51	25.91	27.14
TextCNN	55.37	52.84	58.20	48.38	54.25	45.27	47.13	43.56	87.42	24.34	21.90	27.38
SySeVR	60.56	57.55	66.67	50.63	51.84	49.72	56.66	44.29	87.93	25.34	29.37	22.29
Reveal	58.58	53.58	64.53	45.80	54.37	43.48	47.43	40.14	89.07	26.97	30.00	24.49
VulCNN	58.54	58.14	58.22	58.07	56.17	48.87	52.49	45.71	87.75	33.60	48.64	25.67
TrVD	59.06	57.93	61.76	54.55	56.19	49.13	52.64	46.06	83.47	32.37	27.61	39.13
CodeBERT	62.10	56.76	69.01	48.20	57.96	45.49	50.05	41.68	91.25	24.33	60.38	15.24
UniXCoder	60.91	59.55	63.91	55.74	57.73	48.08	49.76	46.51	90.14	33.44	39.23	29.14
GraphCodeBERT	59.56	59.42	61.62	57.38	57.21	44.02	48.96	39.98	91.16	27.44	56.72	18.10
EPVD	61.74	55.81	69.23	49.75	54.72	49.53	48.02	53.14	87.59	33.18	33.01	33.33
LineVul	58.72	64.92	57.87	74.03	54.32	44.53	47.13	42.21	87.68	30.35	29.05	31.77
DeepSeek-V3	49.83	44.32	51.97	38.64	55.71	38.83	48.55	32.35	69.03	16.39	10.92	32.86
Qwen3-14B	52.02	48.26	55.23	42.86	54.83	37.74	47.04	37.51	69.78	14.87	10.05	28.57
GPT-4o	52.35	55.21	53.68	56.82	53.88	47.50	46.99	48.02	66.09	24.26	15.56	55.10
GPT-5	55.03	61.82	55.08	70.45	52.82	50.90	46.45	56.28	59.17	19.86	54.76	12.13
SIGAR	62.50	66.46	61.50	72.30	57.36	51.49	50.75	52.24	89.83	34.34	41.30	29.38

TABLE II: Performance comparison with baseline methods across three datasets

score. Although these gains are relatively smaller, they further confirm the advantage of leveraging pre-trained representations. However, unlike conventional fine-tuning that require updating a large number of parameters, our method optimizes only lightweight perturbations, making it to achieve improved effective adaptation.

In addition, prompting-based LLM approaches exhibit noticeably lower performance across all datasets, which aligns with observations from prior studies [20], [23], [49], [50]. This limitation can largely stems from the broad, general-purpose training objectives of LLMs, as well as their limited exposure to vulnerability-specific patterns, which are underrepresented in pre-training corpora due to the scarcity of vulnerable code samples. As a result, these models struggle to accurately identify vulnerabilities under zero-shot settings. In contrast, SIGAR achieves the largest improvements over this category, with absolute gains of 12.35% in accuracy and 12.43% in F1 score averaged across the three datasets.

Answer to RQ1: SIGAR consistently outperforms all baseline methods across the three datasets, achieving the best F1 scores and competitive accuracy. It demonstrates superior robustness even under imbalanced settings, and provides more effective vulnerability detection than the evaluated training-from-scratch, PTM-based, and LLM prompting-based approaches.

2) *RQ2: Performance under Data Limited Setting:* The ability to perform effectively with limited training data is a crucial ability for DL-based vulnerability detection, as newly disclosed vulnerabilities often come with only a small number of labeled samples. In such scenarios, models should quickly adapt and generalize from scarce data to identify emerging vulnerability patterns. To evaluate the robustness of SIGAR under data-limited conditions, we conduct experiments using reduced training subsets. Specifically, for each dataset, we randomly sample 10, 50, 100, 500, and 1000 instances from the training set, while keeping the validation and test sets

unchanged. For each sampling size, the experiment is repeated 10 times with different random seeds, and the results on the test set are averaged to ensure statistical reliability.

Figure 3 shows how performance changes across different levels of training data scarcity, measured by accuracy and F1 score. As the results show, all methods² exhibit generally improved performance as the number of training samples increases, reflecting the inherent reliance of deep learning models on sufficient training data for effective representation learning. Across the majority datasets and sample sizes, SIGAR achieves superior results compared to the competing approaches, as indicated by the red curves. This consistent advantage demonstrates its strong generalization capability in low-data regimes. The performance gain can be attributed to its adopted adversarial reprogramming mechanism, which enables efficient adaptation by optimizing only a small set of parameters while effectively leveraging the feature perception capability of the pre-trained backbone model.

Specifically, on the D2A dataset, SIGAR exhibits clear advantages in both accuracy and F1 score, achieving average absolute improvements of 2.57% and 9.08%, respectively, over baseline methods. Remarkably, even with only 500 training samples, it attains an F1 score of 65.53%, outperforming all baselines even when they are trained on the full dataset. A similar trend is observed on the Devign dataset, where SIGAR maintains a dominant position in F1 score across all sample sizes, with an average margin of 6.97% over the competing approaches. In particular, with just 100 training samples, it reaches an F1 score of 46.76%, exceeding the baselines by approximately 7.71 percentage points. Although this advantage gradually narrows as more training data become available, it remains notable, maintaining 4.44% at 1,000 samples.

On the Reveal dataset, SIGAR continues to outperform the baselines, achieving average gains of 2.96% in accuracy

²Since this experiment focuses on training under limited data conditions, the four LLM prompting-based methods which do not involve model training are not included here.

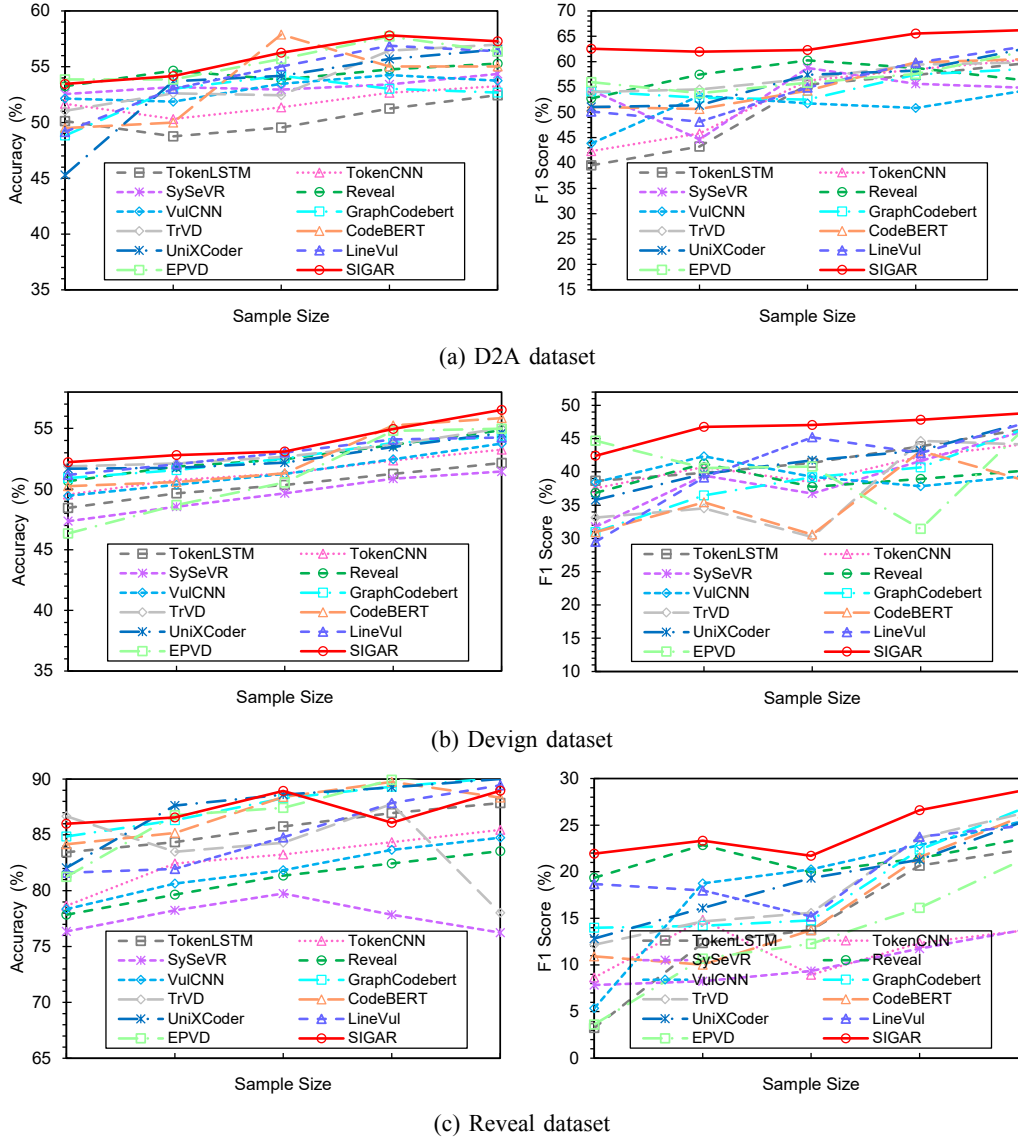


Fig. 3: Performance comparison under data-limited settings

and 7.95% in F1 score. Its superiority persists across all levels of data scarcity, although the margin decreases from 11.32% with only 10 samples to 6.01% at 1,000 samples. Furthermore, when comparing different categories of methods, PTM fine-tuning methods (i.e., CodeBERT, UniXCoder, GraphCodeBERT, LineVul, and EPVD) generally perform better than training-from-scratch models under the limited data conditions, showcasing an average F1 improvement of 1.57% on the Reveal dataset. This observation further highlights the benefits of leveraging pre-trained models in low-resource scenarios, while SIGAR advances this advantage even further.

Answer to RQ2: SIGAR demonstrates superior effectiveness compared to the baseline models under the evaluated data-constrained settings. It exhibits strong generalization ability even with extremely small training sets and maintains a clear advantage as data scale increases, confirming its suitability for data-efficient vulnerability detection.

3) *RQ3: Effect of Adversarial Reprogramming:* Although the preceding experiments demonstrate the overall effectiveness of SIGAR, they cannot directly determine whether the performance improvements originate from adversarial reprogramming, the proposed semantic image representation, or the use of a pre-trained vision backbone. To isolate the contribution of adversarial reprogramming, we compare SIGAR with three controlled classification and adaptation paradigms while keeping the input semantic images and data splits unchanged.

Specifically, **Linear Probe** freezes the ImageNet-pre-trained ViT backbone and trains only a binary classification

TABLE III: Performance with different backbones and perturbation parameter sizes.

Dataset	Backbone	Acc.	F1.	Prec.	Rec.	Perturbation Size	Backbone Size	Ratio	Input Size
D2A	resnet50	59.03	67.94	57.08	83.89	101,376	25.6 Million	0.40%	$224 \times 224 \times 3$
	tf_efficientnet_b4	58.16	66.00	56.52	76.32	384,048	19.0 Million	2.02%	$380 \times 380 \times 3$
	inception_v3	60.74	57.99	64.73	52.53	219,051	23.9 Million	0.92%	$299 \times 299 \times 3$
	vit_base_patch16_384 ($n = 160$)	57.29	61.32	57.69	65.44	27,648	86.9 Million	0.03%	$384 \times 384 \times 3$
	vit_base_patch16_384 ($n = 224$)	59.55	64.43	59.44	70.33	101,376	86.9 Million	0.12%	$384 \times 384 \times 3$
	vit_base_patch16_384 ($n = 384$)	62.50	66.46	61.49	72.30	393,216	86.9 Million	0.45%	$384 \times 384 \times 3$
Devign	resnet50	54.47	56.22	48.11	68.61	-	-	-	-
	tf_efficientnet_b4	54.67	52.92	48.13	58.76	-	-	-	-
	inception_v3	57.12	50.55	50.50	50.39	-	-	-	-
	vit_base_patch16_384 ($n = 160$)	51.21	50.58	45.21	57.40	-	-	-	-
	vit_base_patch16_384 ($n = 224$)	54.81	49.71	48.18	51.35	-	-	-	-
	vit_base_patch16_384 ($n = 384$)	57.36	51.49	50.75	52.24	-	-	-	-
Reveal	resnet50	89.09	32.07	36.91	28.39	-	-	-	-
	tf_efficientnet_b4	89.78	32.20	40.31	26.80	-	-	-	-
	inception_v3	89.61	27.82	37.07	23.40	-	-	-	-
	vit_base_patch16_384 ($n = 160$)	88.43	30.34	33.33	27.83	-	-	-	-
	vit_base_patch16_384 ($n = 224$)	86.47	31.93	29.31	35.05	-	-	-	-
	vit_base_patch16_384 ($n = 384$)	89.83	34.34	41.30	29.38	-	-	-	-

Note: The backbone size, perturbation size, input size, and parameter ratio remain the same for the corresponding model. As these values are already reported in the D2A rows, we denote them with “-” to avoid redundancy.

TABLE IV: Comparison of different classification and adaptation paradigms using the same SIGAR semantic images.

Dataset	Method	Acc.	F1	Prec.	Rec.	Trainable Params.	Ratio
D2A	Linear Probe	-	-	-	-	-	-
	Full Fine-tuning	-	-	-	-	~86.9M	100%
	VulCNN-style CNN	-	-	-	-	-	-
	SIGAR	62.50	66.46	61.50	72.30	393,216	0.45%
Devign	Linear Probe	-	-	-	-	-	-
	Full Fine-tuning	-	-	-	-	~86.9M	100%
	VulCNN-style CNN	-	-	-	-	-	-
	SIGAR	57.36	51.49	50.75	52.24	393,216	0.45%
Reveal	Linear Probe	-	-	-	-	-	-
	Full Fine-tuning	-	-	-	-	~86.9M	100%
	VulCNN-style CNN	-	-	-	-	-	-
	SIGAR	89.83	34.34	41.30	29.38	393,216	0.45%

head, testing whether the original ImageNet features are already sufficient for vulnerability detection. **Full Fine-tuning** updates all ViT backbone parameters and the binary classification head, representing the conventional high-capacity transfer learning paradigm. **VulCNN-style CNN** trains the convolutional neural network classifier adopted by VulCNN on the same SIGAR semantic images, testing whether a conventional CNN trained from scratch can effectively exploit the generated representation. In contrast, SIGAR freezes the pre-trained ViT backbone and optimizes only the adversarial program with output label remapping.

Table IV compares the four paradigms under controlled input settings. The comparison with Linear Probe determines whether a simple classifier built upon frozen ImageNet features is sufficient. The comparison with VulCNN-style CNN evaluates whether the semantic images can be effectively classified by a conventional CNN without cross-domain reprogramming. Full Fine-tuning provides a high-capacity reference by updating the complete ViT model, while SIGAR only optimizes a lightweight adversarial program. Therefore, the results jointly reveal whether adversarial reprogramming provides stronger adaptation than simpler classifiers and whether it achieves a better effectiveness–efficiency trade-off than full

fine-tuning.

Answer to RQ3: By using identical semantic images and controlled training settings, this experiment isolates the contribution of adversarial reprogramming. The comparison evaluates whether SIGAR provides stronger cross-domain adaptation than Linear Probe and VulCNN-style CNN, while requiring substantially fewer trainable parameters than Full Fine-tuning.

4) *RQ4: Effect of Backbone and Parameter Scale:* As described in Section IV-C, SIGAR leverages adversarial reprogramming to adapt a frozen pre-trained vision model for vulnerability detection. While vit_base_patch16_384 serves as the default backbone, other widely adopted vision architectures may offer different representational characteristics. To examine this, we replace the default Vision Transformer [70] with several representative CNN-based backbones, while keeping the rest of our framework unchanged. Specifically, we evaluate three alternative pre-trained backbones, including resnet50, tf_efficientnet_b4, and inception_v3, all sourced from the TIMM library.

Table III reports the performance of these alternative back-

bones, along with their accepted input image sizes, total model parameters (for conventional fine-tuning), perturbation parameter sizes (for adversarial reprogramming), and the corresponding parameter ratios. As observed, `resnet50`, which incorporates residual connections to alleviate gradient vanishing, achieves competitive performance, obtaining F1 scores of 67.94%, 56.22%, and 32.07% on the D2A, Devign, and Reveal datasets, respectively, making it a strong alternative to the Vision Transformer backbone. In contrast, `inception_v3` generally shows inferior performance across all datasets.

To further investigate the impact of perturbation scale, we vary the perturbation region size n (as described in Section IV-C1) for `vit_base_patch16_384`, considering smaller configurations ($n = 160$ and $n = 224$) in addition to the full setting ($n = 384$). The results indicate that even with a substantially reduced number of trainable parameters (e.g., only 27,648 parameters for $n = 160$), SIGAR can still achieve competitive performance, with F1 scores of 61.32%, 50.58%, and 30.34% across the three datasets. In contrast, conventional fine-tuning would require updating approximately 86.9 million parameters, which is over 3,000 times more. Moreover, increasing the perturbation size generally leads to improved performance, although the gains are relatively moderate. This suggests that while larger perturbation spaces offer greater flexibility for model adaptation, SIGAR remains effective even with highly compact parameter configurations.

Answer to RQ4: SIGAR achieves consistently strong performance across different backbone architectures, demonstrating its flexibility in adapting to diverse vision models. In addition, it maintains competitive effectiveness even with a small number of perturbation parameters, while larger perturbation scales provide moderate performance gains. These results highlight the robustness and parameter efficiency of SIGAR’s adversarial reprogramming design.

5) *RQ5: Effect of Semantic Image Construction:* This experiment examines how different strategies for constructing code semantic images influence the performance of SIGAR. As introduced in Section IV-B, our approach integrates semantic embeddings from `Sent2Vec` with three complementary structural embeddings (`DeepWalk`, `Node2Vec`, and `LINE`) to form a three-channel image representation. To assess the contribution of each component, we compare this design with several simplified variants: (i) `Sent2Vec*3`, which uses only semantic features replicated across three channels; (ii) `DeepWalk*3`, `Node2Vec*3`, `LINE*3`, and `w/o Sent2Vec`, each of which relies solely on structural information; and (iii) `(Sent2Vec+Node2Vec)*3`, which combines semantic features with a single structural embedding and replicates the result. In addition, we evaluate an alternative image construction method from `VulCNN` [61], denoted as `VulCNN-Image`. This method embeds PDG nodes using `Sent2Vec` and augments them with graph centrality measures, including degree, Katz, and closeness centrality, to capture node importance, which are then encoded into three image channels. All other experimental

settings remain unchanged for a fair comparison.

TABLE V: Effect of different image generation strategies.

Dataset	Method	Acc	F1	Prec	Rec
D2A	Sent2Vec*3	59.87	61.94	60.84	63.96
	DeepWalk*3	55.38	62.92	55.19	73.15
	Node2Vec*3	57.12	54.86	60.24	50.34
	LINE*3	54.86	65.24	54.10	82.16
	w/o Sent2Vec	57.99	67.12	56.26	83.17
	(Sent2Vec+Node2Vec)*3	58.16	65.91	56.59	78.72
	VulCNN-Image	58.61	61.23	59.30	63.30
	SIGAR	62.50	66.46	63.71	72.30
Devign	Sent2Vec*3	57.28	51.36	50.99	51.74
	DeepWalk*3	52.93	32.23	42.88	25.82
	Node2Vec*3	53.52	32.04	43.90	25.29
	LINE*3	53.84	41.21	45.96	37.35
	w/o Sent2Vec	52.77	34.54	43.09	28.82
	(Sent2Vec+Node2Vec)*3	57.00	49.13	50.48	47.86
	VulCNN-Image	56.49	50.14	49.91	50.36
	SIGAR	57.36	51.49	50.75	52.24
Reveal	Sent2Vec*3	89.46	33.14	38.89	28.87
	DeepWalk*3	84.24	23.53	20.97	26.80
	Node2Vec*3	87.27	21.33	24.03	19.03
	LINE*3	85.12	28.32	25.10	32.47
	w/o Sent2Vec	86.10	25.13	24.51	29.77
	(Sent2Vec+Node2Vec)*3	89.23	30.21	36.50	28.77
	VulCNN-Image	88.43	33.69	35.00	32.47
	SIGAR	89.83	34.34	41.30	29.38

Table V reports the performance of different semantic image construction strategies. Overall, SIGAR consistently achieves the best results across nearly all datasets and metrics, demonstrating the effectiveness of jointly modeling semantic and structural information. Specifically, the variant `Sent2Vec*3`, which relies solely on semantic embeddings, yields relatively competitive performance but fails to capture essential structural dependencies, limiting its ability to fully characterize vulnerability patterns. In contrast, the structural-only variants (`DeepWalk*3`, `Node2Vec*3`, `LINE*3`, and `w/o Sent2Vec`) perform significantly worse, with SIGAR surpassing them by 4.8% in accuracy and 10.1% in F1 on average. This indicates that structural features alone are generally insufficient for accurate vulnerability detection, as they lack fine-grained semantic context. The hybrid variant `(Sent2Vec+Node2Vec)*3` partially alleviates this issue by combining semantic and structural features, leading to noticeable improvements over single-view representations. However, it still underperforms SIGAR, suggesting that relying on a single type of structural embedding limits the diversity of captured relationships. Furthermore, SIGAR outperforms `VulCNN-Image` by 2.1% in accuracy and 2.4% in F1 averaged across the datasets, indicating the advantage of our image construction strategy over their centrality-based one.

Answer to RQ5: SIGAR achieves the best performance when combining semantic and multiple structural embeddings, demonstrating that multi-view semantic image construction is critical. This design effectively captures both code semantics and structural dependencies, leading to more accurate and robust vulnerability detection than simplified variants.

VI. Discussion

In contrast to conventional rule-based vulnerability detection approaches, which often offer supplementary insights such as potential execution paths associated with detected vulnerabilities [71], SIGAR operates as a deep learning-based model that primarily delivers prediction outcomes without explicit interpretability. Prior researches [72], [73] have explored the integration of explainable AI techniques to pinpoint critical statements influencing model predictions. Integrating such interpretability mechanisms, together with adversarial reprogramming approaches, to improve the transparency of SIGAR and support fine-grained vulnerability localization represents a promising avenue for future work.

Another challenge for DL-based vulnerability detection is semantics-preserving code transformations or perturbations, which can compromise model reliability [74]–[77]. Common transformations, such as identifier renaming and lightweight dead-code insertion, can largely be mitigated by the code normalization processing in SIGAR, which reduces sensitivity to superficial syntactic variations. Nonetheless, more advanced adversarial strategies, such as injecting vulnerable patterns into benign code or adding considerable amounts of irrelevant but valid code to vulnerable samples, may still pose risks. Improving the robustness of SIGAR against such sophisticated attacks using reinforcement strategies such as adversarial training [78] and casual learning [79] remain important directions for future work.

Although our experiments are conducted on widely used vulnerability datasets, label noise remains an inherent issue [80], [81]. In addition, such datasets for academic research may not adequately capture the diversity and complexity of real-world industrial code, potentially limiting the practical effectiveness of DL-based models. Furthermore, the samples are typically isolated functions, which can be syntactically incomplete or non-compilable. While we rely on Joern to construct PDGs, parsing failures may occur and adversely affect detection performance. Nevertheless, consistent with prior work, we view the ability to identify vulnerability-related patterns from imperfect or incomplete code as a necessary capability, since developers frequently analyze partial snippets in practice. Future improvements in code parsing techniques may further enhance the robustness and accuracy of the model.

VII. Conclusion

In this paper, we present SIGAR, a data-efficient vulnerability detection framework that reformulates program analysis as a visual learning problem and leverages adversarial

reprogramming to repurpose pre-trained vision models. By transforming source code into multi-channel semantic images that encode both fine-grained semantics and structural dependencies, SIGAR enables effective cross-domain adaptation without requiring costly model fine-tuning. The proposed lightweight reprogramming strategy further reduces the number of trainable parameters, making the framework particularly suitable for scenarios with limited labeled data. Extensive experiments on three real-world datasets demonstrate that SIGAR consistently outperforms a wide range of baseline methods, including training-from-scratch models, PTM-based approaches, and LLM prompting-based techniques. In particular, SIGAR achieves superior performance under both standard and data-limited settings, confirming its strong generalization ability and robustness. Additional analyses show that the proposed multi-view semantic image construction and adversarial reprogramming design are both critical to the effectiveness of the framework. As future work, we plan to enhance SIGAR’s interpretability using explainable AI techniques, improve its robustness against adversarial transformations, and extend it to broader programming languages.

References

- [1] Infer, “Infer: A tool to detect bugs in java and c/c++/objective-c code before it ships,” <https://fbinfer.com>, 2013.
- [2] S. Lipp, S. Banescu, and A. Pretschner, “An empirical study on the effectiveness of static c code analyzers for vulnerability detection,” ser. ISSTA 2022. New York, NY, USA: Association for Computing Machinery, 2022, p. 544–555.
- [3] N. Shirir Harzevili, A. Boaye Belle, J. Wang, S. Wang, Z. M. Jiang, and N. Nagappan, “A systematic literature review on automated software vulnerability detection using machine learning,” *ACM Computing Surveys*, vol. 57, no. 3, pp. 1–36, 2024.
- [4] C. Liang, Q. Wei, J. Du, Y. Wang, and Z. Jiang, “Survey of source code vulnerability analysis based on deep learning,” *Computers & Security*, vol. 148, p. 104098, 2025.
- [5] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, and Z. Chen, “Sysevr: A framework for using deep learning to detect software vulnerabilities,” *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 4, pp. 2244–2258, 2022.
- [6] S. Chakraborty, R. Krishna, Y. Ding, and B. Ray, “Deep learning based vulnerability detection: Are we there yet,” *IEEE Transactions on Software Engineering*, pp. 1–1, 2021.
- [7] X. Cheng, H. Wang, J. Hua, G. Xu, and Y. Sui, “Deepwukong: Statically detecting software vulnerabilities using deep graph neural network,” *ACM Trans. Softw. Eng. Methodol.*, vol. 30, no. 3, apr 2021.
- [8] S. Cao, X. Sun, X. Wu, D. Lo, L. Bo, B. Li, and W. Liu, “Coca: Improving and explaining graph neural network-based vulnerability detection systems,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
- [9] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang *et al.*, “Codebert: A pre-trained model for programming and natural languages,” in *Findings of the association for computational linguistics: EMNLP 2020*, 2020, pp. 1536–1547.
- [10] M. Fu and C. Tantithamthavorn, “Linevul: A transformer-based line-level vulnerability prediction,” in *Proceedings of the 19th International Conference on Mining Software Repositories*, 2022, pp. 608–620.
- [11] J. Zhang, Z. Liu, X. Hu, X. Xia, and S. Li, “Vulnerability detection by learning from syntax-based execution paths of code,” *IEEE Transactions on Software Engineering*, vol. 49, no. 8, pp. 4196–4212, 2023.
- [12] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, and Y. Chen, “Vulnerability Detection with Code Language Models: How Far Are We?,” in *ICSE, CA, USA, May 2025*, pp. 469–481.

- [13] J. Wang, K. Zheng, B. Wu, C. Wu, Y. Yao, J. Gao, and M. Yang, "Ctx-coder: Cross-attention architectures empower llms for long-context vulnerability detection," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 2, 2026, pp. 1159–1167.
- [14] Y. Chen, Y. Huang, X. Chen, P. Shen, and L. Yun, "Gptvd: vulnerability detection and analysis method based on llm's chain of thoughts," *Automated Software Engineering*, vol. 33, no. 1, p. 3, 2026.
- [15] Y. Nong, R. Fang, G. Yi, K. Zhao, X. Luo, F. Chen, and H. Cai, "Vgx: Large-scale sample generation for boosting learning-based software vulnerability analyses," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
- [16] Y. Nong, H. Yang, F. Chen, and H. Cai, "Vinj: An automated tool for large-scale software vulnerability data generation," in *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, ser. FSE 2024. New York, NY, USA: Association for Computing Machinery, 2024, p. 567–571. [Online]. Available: <https://doi.org/10.1145/3663529.3663800>
- [17] S. S. Daneshvar, Y. Nong, X. Yang, S. Wang, and H. Cai, "Vulscriber: Exploring rag-based vulnerability augmentation with llms," *ACM Trans. Softw. Eng. Methodol.*, Aug. 2025, just Accepted. [Online]. Available: <https://doi.org/10.1145/3760775>
- [18] X. Yang, S. Wang, Y. Li, and S. Wang, "Does data sampling improve deep learning-based vulnerability detection? yeas! and nays!" in *ICSE*, 2023, pp. 2287–2298.
- [19] R. Croft, M. A. Babar, and M. Kholoosi, "Data quality for software vulnerability datasets," in *Proceedings of the 2023 IEEE/ACM International Conference on Software Engineering (ICSE '23)*, 2023, pp. 1–1.
- [20] H. Shu, M. Fu, J. Yu, D. Wang, C. Tantithamthavorn, J. Chen, and Y. Kamei, "Evaluating large language models for multilingual vulnerability detection at dual granularities," *Empirical Software Engineering*, vol. 31, no. 4, p. 110, 2026.
- [21] X.-C. Wen, X. Wang, Y. Chen, R. Hu, D. Lo, and C. Gao, "From function to repository: Towards repository-level evaluation of software vulnerability detection," *IEEE Transactions on Software Engineering*, 2026.
- [22] F. Weissberg, L. Pirch, E. Imgrund, J. Möller, T. Eisenhofer, and K. Rieck, "Llm-based vulnerability discovery through the lens of code metrics," in *International Conference on Software Engineering*, 2026, pp. 1–13.
- [23] D. S. Ouchebara and S. Dupont, "Llama-based source code vulnerability detection: Prompt engineering vs fine tuning," in *European Symposium on Research in Computer Security*. Springer, 2025, pp. 289–308.
- [24] R. Widyasari, M. Weyssow, I. C. Irsan, H. W. Ang, F. Liauw, E. L. Ouh, L. K. Shar, H. J. Kang, and D. Lo, "Let the trial begin: A mock-court approach to vulnerability detection using llm-based agents," in *48th International Conference on Software Engineering*. IEEE, 2026, pp. 1–12.
- [25] X.-C. Wen, Y. Yang, C. Gao, Y. Xiao, and D. Ye, "Boosting vulnerability detection of llms via curriculum preference optimization with synthetic reasoning data," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025, pp. 8935–8949.
- [26] G. F. Elsayed, I. Goodfellow, and J. Sohl-Dickstein, "Adversarial reprogramming of neural networks," *ICLR*, 2019.
- [27] L. Chen, Y. Fan, and Y. Ye, "Adversarial reprogramming of pretrained neural networks for fraud detection," in *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, 2021, pp. 2935–2939.
- [28] P. Neekhara, S. Hussain, J. Du, S. Dubnov, F. Koushanfar, and J. McAuley, "Cross-modal adversarial reprogramming," in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2022, pp. 2427–2435.
- [29] L. Chen, X. Li, and D. Wu, "Adversarially reprogramming pretrained neural networks for data-limited and cost-efficient malware detection," in *Proceedings of the 2022 SIAM International Conference on Data Mining (SDM)*. SIAM, 2022, pp. 693–701.
- [30] A. Smaili, Y. Zhang, D. E. Mekkaoui, M. A. Midoun, M. Z. Talhaoui, M. Hamidaoui, and W. Kong, "A transformer-based framework for software vulnerability detection using attention-driven convolutional neural networks," *Engineering Applications of Artificial Intelligence*, vol. 160, p. 111859, 2025.
- [31] K. Napier, T. Bhowmik, and S. Wang, "An empirical study of text-based machine learning models for vulnerability detection," *Empirical Software Engineering*, vol. 28, no. 2, p. 38, Feb 2023.
- [32] R. Russell, L. Kim, L. Hamilton, T. Lazovich, J. Harer, O. Ozdemir, P. Ellingwood, and M. McConley, "Automated vulnerability detection in source code using deep representation learning," in *2018 17th IEEE international conference on machine learning and applications (ICMLA)*. IEEE, 2018, pp. 757–762.
- [33] D. Zou, S. Wang, S. Xu, Z. Li, and H. Jin, "muvuldeepecker: A deep learning-based system for multiclass vulnerability detection," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 5, pp. 2224–2236, 2021.
- [34] L. Wang, G. Lu, X. Chen, X. Dai, and J. Qiu, "Sift: enhance the performance of vulnerability detection by incorporating structural knowledge and multi-task learning," *Automated Software Engineering*, vol. 32, no. 2, p. 38, Apr 2025.
- [35] S. Yang, J. Ren, J. Li, and D. Zhang, "Vuldiac: Vulnerability detection and interpretation based on augmented cfg and causal attention learning," *Journal of Systems and Software*, vol. 231, p. 112595, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S016412122500264X>
- [36] S. Cao, X. Sun, L. Bo, Y. Wei, and B. Li, "Bgnn4vd: Constructing bidirectional graph neural-network for vulnerability detection," *Information and Software Technology*, vol. 136, p. 106576, 2021.
- [37] Z. Tian, B. Tian, J. Lv, Y. Chen, and L. Chen, "Enhancing vulnerability detection via ast decomposition and neural sub-tree encoding," *Expert Systems with Applications*, vol. 238, p. 121865, 2024.
- [38] X.-C. Wen, C. Gao, S. Gao, Y. Xiao, and M. R. Lyu, "Scale: Constructing structured natural language comment trees for software vulnerability detection," in *Proceedings of the 33rd ACM SIGSOFT international symposium on software testing and analysis*, 2024, pp. 235–247.
- [39] Y. Li, S. Wang, and T. N. Nguyen, "Vulnerability detection with fine-grained interpretations," in *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 292–303.
- [40] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, "Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks," *Advances in neural information processing systems*, vol. 32, 2019.
- [41] X. Sun, M. Zhou, S. Cao, X. Wu, L. Bo, D. Wu, B. Li, and Y. Xiang, "HgtJIT: Just-in-Time Vulnerability Detection Based on Heterogeneous Graph Transformer," *IEEE Transactions on Dependable and Secure Computing*, no. 01, pp. 1–17, Jul. 5555. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/TDSC.2025.3586669>
- [42] M. Shao, Y. Ding, J. Cao, and Y. Li, "Graphfvd: Property graph-based fine-grained vulnerability detection," *Computers & Security*, vol. 151, p. 104350, 2025.
- [43] X. Li, A. Bhattacharjya, Q. Li, M. Zhou, R. Wisniewski, and G. Bazydło, "A novel method for vulnerability detection based on fusion and hyperbolic neural network graphs," *IEEE Transactions on Software Engineering*, 2026.
- [44] Y. Huang, M. He, X. Wang, and J. Zhang, "Hevuld: A static vulnerability detection method using heterogeneous graph code representation," *IEEE Transactions on Information Forensics and Security*, 2024.
- [45] Y. Zhao, L. Gong, Z. Huang, Y. Wang, M. Wei, and F. Wu, "Coding-ptms: How to find optimal code pre-trained models for code embedding in vulnerability detection?" in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, ser. ASE '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 1732–1744. [Online]. Available: <https://doi.org/10.1145/3691620.3695539>
- [46] J. Lin, D. Mohaisen *et al.*, "From large to mammoth: A comparative evaluation of large language models in vulnerability detection," in *NDSS*, 2025.
- [47] I. Kalouptoglou, M. Siavvas, A. Ampatzoglou, D. Kehagias, and A. Chatzigeorgiou, "Transfer learning for software vulnerability prediction using transformer models," *J. Syst. Softw.*, vol. 227, no. C, Jun. 2025.
- [48] X. Zhou, S. Cao, X. Sun, and D. Lo, "Large language model for vulnerability detection and repair: Literature review and the road ahead," *ACM Trans. Softw. Eng. Methodol.*, Dec. 2024. [Online]. Available: <https://doi.org/10.1145/3708522>
- [49] Y. Guo, C. Patsakis, Q. Hu, Q. Tang, and F. Casino, "Outside the comfort zone: Analysing llm capabilities in software vulnerability detection," in *European symposium on research in computer security*. Springer, 2024, pp. 271–289.

- [50] X. Yin, C. Ni, and S. Wang, "Multitask-based evaluation of open-source llm on software vulnerability," *IEEE Trans. Softw. Eng.*, vol. 50, no. 11, p. 3071–3087, Nov. 2024.
- [51] G. Lu, X. Ju, X. Chen, W. Pei, and Z. Cai, "Grace: Empowering llm-based software vulnerability detection with graph structure and in-context learning," *J. Syst. Softw.*, vol. 212, no. C, Jun. 2024. [Online]. Available: <https://doi.org/10.1016/j.jss.2024.112031>
- [52] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "Llms cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks," in *2024 IEEE symposium on security and privacy (SP)*. IEEE, 2024, pp. 862–880.
- [53] A. Lekssays, H. Mouhcine, K. Tran, T. Yu, and I. Khalil, "{LLMxCPG}:{Context-Aware} vulnerability detection through code property {Graph-Guided} large language models," in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 489–507.
- [54] F. Mao, Y. Yuan, X. Du, L. Gao, and Z. Du, "Mpda: a data augmentation approach to improve deep learning for software vulnerability detection," *Empirical Softw. Engg.*, vol. 30, no. 5, Jul. 2025. [Online]. Available: <https://doi.org/10.1007/s10664-025-10698-y>
- [55] S. S. Daneshvar, D. Tan, S. Wang, and C. Leung, "A study on mixup-inspired augmentation methods for software vulnerability detection," in *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering*, ser. EASE '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 616–621. [Online]. Available: <https://doi.org/10.1145/3756681.3757017>
- [56] Y. Nong, Y. Ou, M. Pradel, F. Chen, and H. Cai, "Generating realistic vulnerabilities via neural code editing: An empirical study," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ser. ESEC/FSE 2022. New York, NY, USA: Association for Computing Machinery, 2022, p. 1097–1109.
- [57] —, "Vulgen: Realistic vulnerability generation via pattern mining and deep learning," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2527–2539.
- [58] H. Yong, Z. Li, M. Pan, T. Zhang, J. Zhao, and X. Li, "GVI: Guided Vulnerability Imagination for Boosting Deep Vulnerability Detectors," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2025, pp. 2867–2879. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/ICSE55347.2025.00214>
- [59] G. Yi, Y. Nong, M. Li, and H. Cai, "Exploring and improving real-world vulnerability data generation via prompting large language models," in *48th International Conference on Software Engineering*. IEEE, 2026, pp. 1–12.
- [60] Q. Li, L. Chen, S. Jing, and D. Wu, "Knowledge distillation on cross-modal adversarial reprogramming for data-limited attribute inference," in *Companion Proceedings of the ACM Web Conference 2023*, ser. WWW '23 Companion. New York, NY, USA: Association for Computing Machinery, 2023, p. 65–68. [Online]. Available: <https://doi.org/10.1145/3543873.3587313>
- [61] Y. Wu, D. Zou, S. Dou, W. Yang, D. Xu, and H. Jin, "Vulcnn: An image-inspired scalable vulnerability detection system," in *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*, 2022, pp. 2365–2376.
- [62] M. Pagliardini, P. Gupta, and M. Jaggi, "Unsupervised Learning of Sentence Embeddings using Compositional n-Gram Features," in *NAACL 2018 - Conference of the North American Chapter of the Association for Computational Linguistics*, 2018.
- [63] B. Perozzi, R. Al-Rfou, and S. Skiena, "Deepwalk: Online learning of social representations," in *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2014, pp. 701–710. [Online]. Available: <https://doi.org/10.1145/2623330.2623732>
- [64] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 855–864. [Online]. Available: <https://doi.org/10.1145/2939672.2939754>
- [65] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, and Q. Mei, "Line: Large-scale information network embedding," in *Proceedings of the 24th International Conference on World Wide Web (WWW)*, 2015, pp. 1067–1077. [Online]. Available: <https://doi.org/10.1145/2736277.2741093>
- [66] Y. Zheng, S. Pujar, B. Lewis, L. Buratti, E. Epstein, B. Yang, J. Laredo, A. Morari, and Z. Su, "D2a: A dataset built for ai-based vulnerability detection methods using differential analysis," in *Proceedings of the 43rd International Conference on Software Engineering: Software Engineering in Practice*, ser. ICSE-SEIP '21. IEEE Press, 2021, p. 111–120.
- [67] Y. Zhou, S. Liu, J. Siow, X. Du, and Y. Liu, *Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks*. Red Hook, NY, USA: Curran Associates Inc., 2019.
- [68] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu *et al.*, "Graphcodebert: Pre-training code representations with data flow," *arXiv preprint arXiv:2009.08366*, 2020.
- [69] D. Guo, S. Lu, N. Duan, Y. Wang, M. Zhou, and J. Yin, "Unixcoder: Unified cross-modal pre-training for code representation," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2022, pp. 7212–7225.
- [70] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," *ICLR*, 2021.
- [71] X. Cheng, X. Nie, N. Li, H. Wang, Z. Zheng, and Y. Sui, "How about bug-triggering paths? - understanding and characterizing learning-based vulnerability detectors," *IEEE Transactions on Dependable and Secure Computing*, pp. 1–18, 2022.
- [72] B. Cheng, M. Zhao, K. Wang, M. Wang, G. Bai, R. Feng, and *et al.*, "Beyond fidelity: Explaining vulnerability localization of learning-based detectors," *ToSEM*, 2024.
- [73] Z. Chu, Y. Wan, Q. Li, Y. Wu, H. Zhang, Y. Sui, G. Xu, and H. Jin, "Graph neural networks for vulnerability detection: A counterfactual explanation," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 389–401.
- [74] Y. Chen, W. Sun, C. Fang, Z. Chen, Y. Ge, T. Han, Q. Zhang, Y. Liu, Z. Chen, and B. Xu, "Security of language models for code: A systematic literature review," *ACM Transactions on Software Engineering and Methodology*, 2025.
- [75] X. Du, M. Wen, H. Wang, Z. Wei, and H. Jin, "Statement-level adversarial attack on vulnerability detection models via out-of-distribution features," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 3009–3032, 2025.
- [76] L. Huang, W. Sun, and M. Yan, "Iterative generation of adversarial example for deep code models," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. IEEE, 2025, pp. 2213–2224.
- [77] Y. Jiang, S. Huang, C. Treude, X. Su, and T. Wang, "Shield broken: Black-box adversarial attacks on llm-based vulnerability detectors," *IEEE Transactions on Software Engineering*, vol. 52, no. 1, pp. 246–265, 2026.
- [78] Y. Wang, Y. Chen, Y. Zhao, Z. Gong, J. Chen, and D. Hao, "Mutual learning-based framework for enhancing robustness of code models via adversarial training," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 1484–1496.
- [79] J. Ye, Z. Li, X. Tang, D. Zou, S. Xu, W. Qiang, and H. Jin, "A causal learning framework for enhancing robustness of source code models," *Proceedings of the ACM on Software Engineering*, vol. 2, no. FSE, pp. 2641–2664, 2025.
- [80] Y. Hu, S. Wang, Y. Ji, Y. Wu, and D. Zou, "Vulnoise: Outlier detection to reduce label noises for effective vulnerability detection," *ACM Transactions on Software Engineering and Methodology*, 2026.
- [81] N. Risse, J. Liu, and M. Böhme, "Top score on the wrong exam: On benchmarking in machine learning for vulnerability detection," *Proc. ACM Softw. Eng.*, vol. 2, no. ISSTA, Jun. 2025. [Online]. Available: <https://doi.org/10.1145/3728887>